Compiling FFI-heavy OCaml to WebAssembly: An Experience Report on MOPSA

Reda Boudrouss
Sorbonne University
contact@rboud.com

July 13, 2026

Abstract

Many large OCaml applications do not consist of OCaml code alone. They depend on native C and C++ libraries accessed through the OCaml foreign function interface (FFI), where bindings often manipulate OCaml values directly and rely on the runtime's data representation and memory layout. This coupling is what makes such applications hard to bring to the browser. How to port them together with their entire native dependency stack remains largely unexplored.

OCaml programs run either as machine code from the native compiler or as bytecode interpreted by the OCaml runtime. Neither runs in a browser, so the existing browser backends translate the program ahead of time, to JavaScript (`js_of_ocaml`) or to WebAssembly (`wasm_of_ocaml`). All of them compile the OCaml code alone and leave the C and C++ behind the FFI to be reimplemented by hand.

Rather than translating the program, we compile the OCaml runtime itself to WebAssembly with Emscripten, with the complete native dependency stack, into a single statically linked module, and interpret the unmodified bytecode on top of it.

This paper reports on our experience doing so for MOPSA, a static analyzer that combines OCaml with LLVM/Clang, GMP, MPFR, Apron, and CamlIDL-generated bindings, running it entirely inside a web browser. Linking the runtime supplies the C primitives the FFI stubs call. The bytecode's external calls are bound through a static primitive table, the mechanism behind OCaml's statically linked (custom) executables, instead of the dynamic loading the runtime performs at startup. Neither the hand-written FFI code (MOPSA's `Clang_to_ml.cc`) nor the CamlIDL-generated stubs are modified.

Introduction

Large OCaml applications rarely consist of OCaml code alone. They rest on native libraries reached through the foreign function interface (FFI), and those bindings allocate OCaml blocks, read their tags and fields, and call back into the runtime. Porting an application of this kind to the browser raises the question of how to preserve the interaction between OCaml and its dependencies.

An OCaml program is either compiled to native machine code or reduced to bytecode that the OCaml runtime interprets. To run in a browser, where neither is available, the existing backends translate the program ahead of time to a web target: `js_of_ocaml`[6] and `wasm_of_ocaml` from the bytecode (to JavaScript and to WebAssembly respectively), `wasocaml` from the Flambda IR (to WebAssembly). All of them compile the OCaml code and leave the C and C++ behind, so the FFI stubs call runtime functions such as `caml_alloc` and `caml_callback` that do not exist in the resulting module. Bridging a native component across the boundary then means hand-writing JavaScript/Wasm glue for it, and the effort grows with every binding.

We contribute (i) a general, reproducible recipe to run FFI-heavy OCaml on Wasm by compiling the bytecode runtime rather than the code, requiring no per-binding glue. (ii) Three runtime/ABI portability hazards we encountered. (iii) A performance comparison against native MOPSA and `js_of_ocaml` that quantifies the cost of interpretation and finds the Wasm build competitive with `js_of_ocaml` where both can run.

The project we compiled is MOPSA. A fully client-side live build is available at https://mopsawasm.rboud.com/.

The problem we address

MOPSA is a static analyzer based on abstract interpretation for C and Python [1], written mostly in OCaml. Beneath it sit GMP, MPFR, Zarith, and Apron for the numerical domains [3], together with LLVM/Clang to parse C, all bound through the FFI. Roughly 655 of the ~1435 primitives the analyzer needs are CamlIDL-generated stubs [5]. One internal file, `Clang_to_ml.cc` (~5000 lines), includes the Clang headers and `<caml/*.h>` at the same time. It drives Clang to parse a source file and then allocates OCaml values from the resulting AST directly inside the GC heap, using the `CAMLparam`/`CAMLlocal`/`CAMLreturn` macros.

Existing solutions do not cover our use case. They are well suited to pure-OCaml code, or to code whose few native dependencies already have a JavaScript reimplementation (e.g. `zarith_stubs_js`). In our case they do not expose FFI functions that we could link our Wasm-compiled `Clang_to_ml.cc` against[1], and rewriting the entire 5000-line `Clang_to_ml.cc` file, along with all the CamlIDL-generated stubs, in JavaScript is not viable.

Compiling the bytecode runtime

The missing `caml_*` symbols are provided by the OCaml runtime itself[2]. Once the runtime is included, everything left to compile is C and C++, so we can rely on Emscripten alone

[1]These backends move OCaml values out of the linear-memory layout the FFI assumes (WasmGC), so a C stub that reads a value's fields or tag by pointer cannot reach them at all. Exposing these functions is not a solution either.

[2]We chose to compile the latest OCaml 4 release (4.14.2 when we started), since our target was wasm32 and OCaml 5 dropped support for 32-bit.

[7] and avoid stitching together modules produced by two compilers with two different memory models. The plan is therefore to compile the OCaml to bytecode, compile the runtime plus every native dependency to Wasm with Emscripten, link that native bundle statically, and interpret the bytecode on top.

OCaml normally resolves the C code behind each `external` dynamically. At startup it `dlopen`s the native libraries and `dlsym`s each primitive by name to fill a table of function pointers. Emscripten can emulate dynamic module loading, but that splits the binary into several Wasm modules and complicates the build. A fully static approach is available instead.

Before searching `.so` files, `lookup_primitive` first consults a built-in table (`caml_builtin_cprim[]` and `caml_names_of_builtin_cprim[]`) that `ocamlc -custom` generates as a `prims.c` file. We generate our own `prims.c` as a superset of every primitive the bytecode calls (the runtime core, `unix`, `str`, bigarray/int64, and the CamlIDL Apron/GMP stubs), disable the `dlopen` branch, and let `ERROR_ON_UNDEFINED_SYMBOLS=1` turn a missing primitive into a link-time failure rather than an `unknown C primitive` trap in the browser.

Portability hazards

While testing, we encountered hazards where code compiles and links correctly yet is wrong, because Wasm (wasm32 in particular) is not the native target it was written for.

A runtime macro that is unsound on wasm32. The first browser run trapped with "index out of bounds", traced to OCaml 4.14's `Tag_val`, which reads the block header at offset `-sizeof(value)`. Since `sizeof` is unsigned, that offset is `0xFFFFFFFC`, not `-4`. On native ILP32 the address wraps in a 32-bit `add` and lands on `val-4`, which is why the issue never surfaces upstream on native targets. On wasm32 the backend can fold the non-negative constant into a bounds-checked unsigned `i32.load` offset that does not wrap, so the access lands ~4,GiB away and traps. The fold is itself backend-dependent, since `emcc`/clang 22 traps while `wasi-sdk` clang 18 does not. We fix it by casting `sizeof(value)` to a signed `int` before negating, so the subscript carries a genuine `-4` instead of the unsigned `0xFFFFFFFC`. A negative displacement cannot be folded into the load's static offset, so the backend keeps the wrapping `i32.add` and computes `val - 4`.

Architecture-dependent ABI in the analyzed program. Porting the stack to wasm32 also retargeted MOPSA's embedded Clang from x86-64 to 32-bit, so the sources it parses are now typed for a 32-bit ABI. There `va_list` is a scalar `void*` passed *by reference* to `__builtin_va_start`, so Clang hands MOPSA's type translator an `LValueReferenceType`, a case it lacked because on x86-64 `va_list` is an array that decays to a pointer. That missing case broke the translation of CPython's C sources. We added the missing `LValueReferenceType` case to `Clang_to_C.ml`'s type translator, modelling the reference as the ABI-equivalent pointer, after which CPython's sources parse correctly.

A Wasm ABI limitation that threatens soundness. WebAssembly has no FPU rounding-mode control, so everything is round-to-nearest and `fesetround` is a no-op. This threatens soundness on two fronts. Apron's interval arithmetic relies on directed rounding, which we sidestep by compiling every domain with `NUM_MPQ` (bounds computed exactly with GMP rationals) and stubbing out the FPU probe, though a residual unsoundness remains where floats reach Apron's API before conversion to rationals. MOPSA's own float handling (`floats_round.c`) depends on the same rounding-mode control, and there we still have no satisfying fix. Widening every interval to a safe over-approximation keeps the analysis sound but coarse. Unlike the first two, this is not a 32-bit quirk but a property of Wasm itself, it affects soundness rather than only portability.

Performance

We compared the Wasm build against native MOPSA on a corpus of fourteen C, Python, and universal files, over 100 repetitions each in Node and in a headless browser.[3] Interpreted bytecode on Wasm is, as expected, slower than native compiled code. In steady state, once V8 has recompiled the module with its optimizing tier (Wasm code first runs through a fast baseline compiler and is re-compiled by the optimizing one once it is hot), analysis runs about 10 times slower than native on the C files and about 13 times slower on Python.

Our `js_of_ocaml` baseline is Try-MOPSA [2], a scaled-down build of MOPSA compiled to JavaScript with `js_of_ocaml`. Since `js_of_ocaml` translates only the OCaml code, Try-MOPSA cannot carry the native C and C++ dependencies. It therefore supports no C analysis and, for its relational domain, replaces the native Apron with VPL [4], a pure-OCaml library of convex polyhedra.
The two are close on a single cold run, but a fresh analysis needs a fresh OCaml state, and `js_of_ocaml` keeps state and code together in one JavaScript realm (its global environment), so obtaining a clean state means recreating that realm from scratch and paying the JIT warm-up cost again. WebAssembly instead re-instantiates a module that V8 has already optimized, so on repeated analyses the Wasm build pulls ahead, by roughly 1.3 times on Python under Node and 3.4 times in the browser. It also instantiates about 6 times faster, 63 ms against 359 ms under Node, since there is no 22 MB JavaScript bundle to parse. On the full C and Apron workload, Try-MOPSA cannot run at all, for the reason above, whereas the Wasm build does.

Discussion

None of this is specific to MOPSA. Any FFI-heavy OCaml application whose native stack is impractical to reimplement can be carried to the browser the same way, and will meet the same class of silent Wasm-versus-native divergences.

[3] All timings were collected on an AMD Ryzen 7 1700X (8 cores, 16 threads) with 24 GB of RAM running Arch Linux (Linux 7.1.2), under Node.js 22.22.3 (V8 12.4) and headless Chromium 150.

Most of the recipe is mechanical, and we believe much of it could be automated. Emscripten's build tooling has matured to the point where several of our native dependencies compiled with no patch at all, and the primitive-table generation and static-linking steps are systematic. One can imagine a dedicated build target, for instance in dune, that compiles a whole OCaml project together with its native stack to Wasm from sources alone, given an opam switch with sources, leaving only genuinely native components such as LLVM/Clang to case-by-case work. Whether such a reusable path is worth building as a shared effort, at least until a WasmGC backend such as `wasm_of_ocaml` can interoperate with Emscripten-compiled C, is a question we would like to raise with the community. How far the approach carries to OCaml 5 is a further open question, and an early experiment is encouraging, since the `Tag_val` issue is already fixed in the latest 5.x and the remaining wasm-hostile behavior appears possible to disable.

Availability. The port targets OCaml 4.14 (LLVM/Clang 9, GMP 6.1.2, MPFR 4.2.2) and runs entirely client-side. Sources are at https://github.com/rboudrouss/mopsa-wasm.

Acknowledgements

We thank Antoine Miné for guiding us through the MOPSA codebase and supporting this work throughout, and Raphaël Monat for Try-MOPSA [2], an all-`js_of_ocaml` build of MOPSA. Try-MOPSA produced the `js_of_ocaml`/VPL configuration we compare against and was invaluable in getting our interactive mode working in the browser. The port also builds on Vincent Chan's `ocaml-wasm` [8], which supplied the original Emscripten tweaks, on binji's LLVM-to-wasm fork [9], and on a Stack Overflow answer [10] that identified Emscripten-compatible GMP and MPFR versions.

References

[1] M. Journault, A. Miné, R. Monat, and A. Ouadjaout. *Combinations of Reusable Abstract Domains for a Multilingual Static Analyzer.* VSTTE 2019, LNCS 12031.

[2] R. Monat. *Try-Mopsa: Relational Static Analysis in Your Pocket.* VMCAI 2026. arXiv:2509.13128. https://arxiv.org/abs/2509.13128.

[3] B. Jeannet and A. Miné. *Apron: A Library of Numerical Abstract Domains for Static Analysis.* CAV 2009, LNCS 5643.

[4] S. Boulmé, A. Maréchal, D. Monniaux, M. Périn, and H. Yu. *The Verified Polyhedron Library: an Overview.* SYNASC 2018.

[5] X. Leroy. *CamlIDL User's Manual.* https://github.com/xavierleroy/camlidl.

[6] J. Vouillon and V. Balat. *From bytecode to JavaScript: the Js_of_ocaml compiler.* Software: Practice and Experience, 44(8), 2014.

[7] A. Zakai. *Emscripten: an LLVM-to-JavaScript Compiler.* OOPSLA 2011 (companion), ACM.

[8] V. Chan. *ocaml-wasm.*. https://github.com/vincentdchan/ocaml.

[9] binji. *Building LLVM/Clang to WebAssembly.*. https://gist.github.com/binji/b7541f9740c21d7c6dac95cbc9ea6fca.

[10] Stack Overflow. *Compiling GMP/MPFR with Emscripten.* https://stackoverflow.com/a/43583154.